

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL2 - CONCEPT MAPPING

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	C01	Assessment:	ASSESSMENT TOOL 2- CONCEPT MAPPING
Weightage:	35%	Total Marks:	25
C01 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)		
Student Name:	KAVALI BALAJI	Reg.No.:	192525402
Department:	AIML	Date:	14-07-2026

Instruction to Students

1. Answer two questions as per the Instruction.
2. Each question carries 12.5 mark.
3. Only the appropriate Tools can be used
4. Time allotted for the exam is 60 min

ANNEXURE A

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
 2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.
 3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.
 4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.
 5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.
-

Code of Conduct:

I G. Naveen Kumar Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

Criteria	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)
Q1					
Q2					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

SIMATS ENGINEERING

Assessment Tool 2 – Concept Mapping
Computer Architecture for Multicore Systems (CSA1205)

Name	KAVALI BALAJI
Register No.	192525402
Course Outcome	CO1 (BL4) – CPU, Memory, Bus Structures and I/O Components

Q1. CPU, Memory, I/O and Bus Structures

Explanation

The computer system architecture is composed of the CPU, memory, and I/O devices. The CPU itself includes the ALU, Control Unit, and Registers, which together perform arithmetic, control signal generation, and temporary data storage. Memory is organized as cache (fast, small) and RAM (larger, slower), forming a hierarchy that balances speed and capacity. All three units communicate through bus structures, which carry data, address, and control signals.

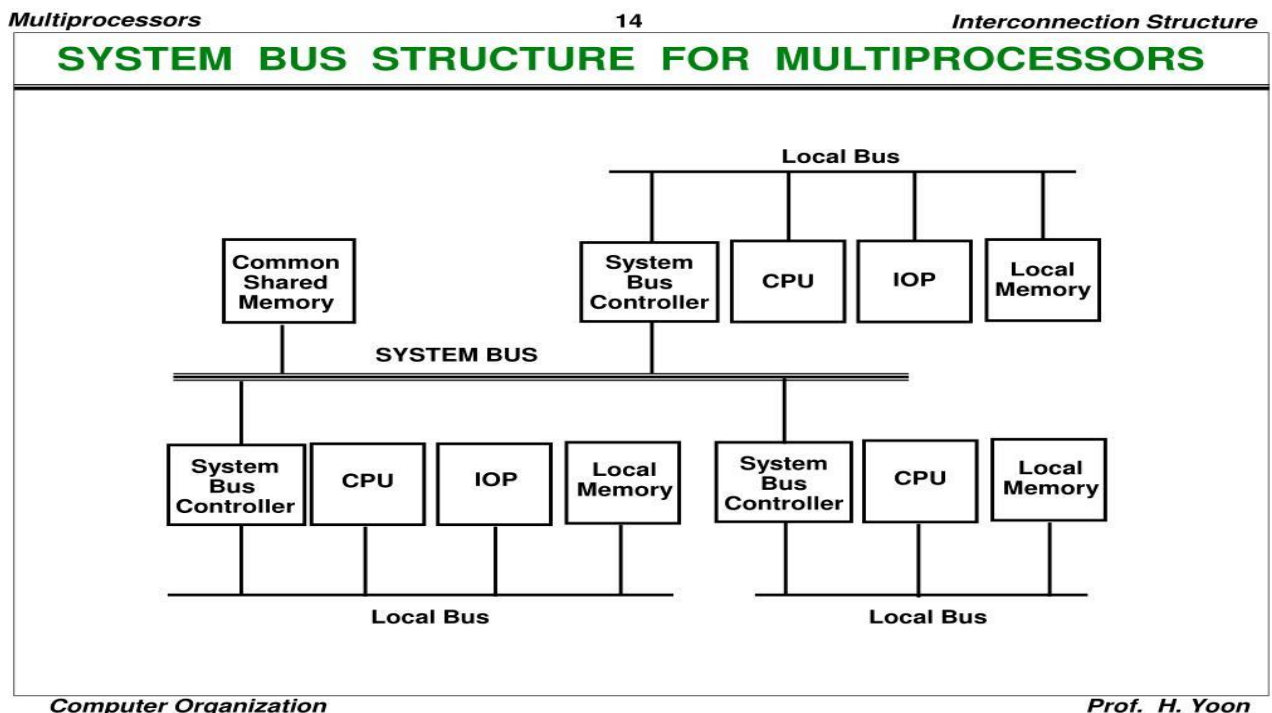
A single-bus structure allows only one transfer between components at a time, so as more devices contend for the bus, delay and bottlenecks increase – this is a key point of contention during peak load. Multi-bus and hierarchical bus structures separate traffic across dedicated paths (e.g., a fast local bus for CPU-memory and a slower bus for I/O), allowing parallel transfers, reducing contention, and improving overall throughput and performance.

Relationship

- CPU ↔ Memory → Reads/Writes Data
- CPU ↔ I/O Devices → Receives Input & Sends Output
- Bus Structure ↔ CPU, Memory, I/O → Connects and Enables Communication Between All Components

This content is suitable for drawing a concept map with "Computer System" as the central node and CPU, Memory, I/O, and Bus Structure as the four main branches.

Concept map



CONCLUSION

The CPU processes data, memory stores information, I/O devices handle user interaction, and buses enable communication among all components. Together, they form the basic architecture of a computer system.

Q2. Instruction Execution Cycle and CPU Performance

Explanation

The instruction execution cycle begins with Fetch, proceeds to Decode, and then Execute. Each of these stages contributes to the total execution time of a program. Execution time is determined by two key parameters: CPI (cycles per instruction) and the number of clock cycles required. A higher CPI directly increases execution time for the same clock speed.

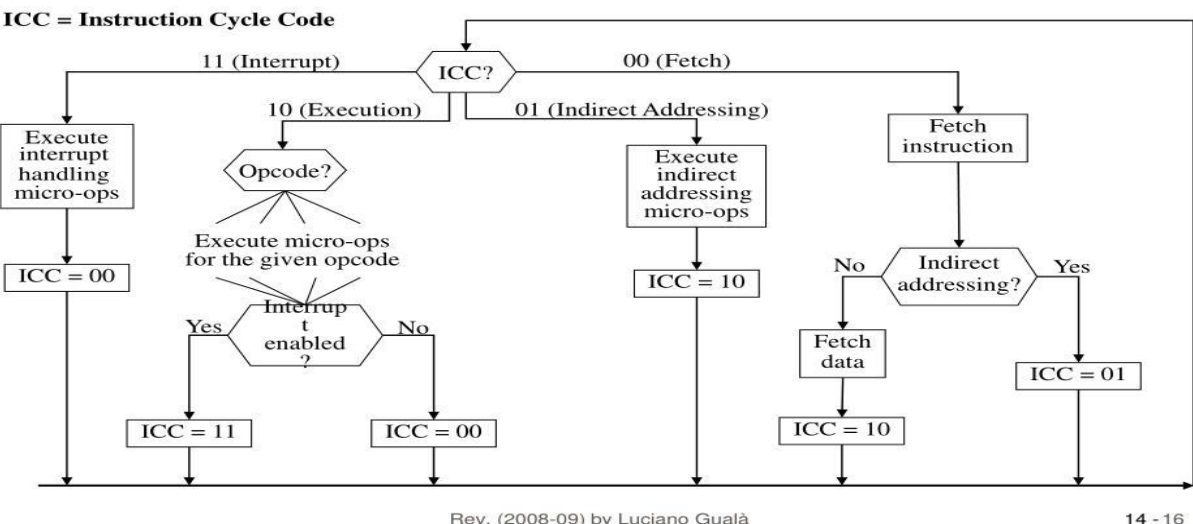
CPI itself is increased by factors such as memory access time (slow memory reads/writes stall the pipeline) and instruction complexity (multi-step or multi-operand instructions take longer to decode and execute). These factors can trigger pipeline stalls, which in turn raise the effective CPI further, creating a feedback relationship between complexity, stalls, and performance. Both execution time and CPI are used together in benchmarking, which evaluates and compares real-world CPU performance across different workloads and architectures.

Relationship

- The Instruction Execution Cycle (Fetch → Decode → Execute → Memory Access → Write Back) defines how the CPU processes instructions.
- CPU performance depends on how quickly and efficiently each stage of the execution cycle is completed.
- Faster clock speed, efficient pipelining, larger cache memory, and fewer execution delays improve CPU performance.
- Reduced instruction execution time results in higher system throughput and better overall performance.
- Thus, an optimized instruction execution cycle directly enhances CPU efficiency and speed.

Concept map

A simplified flow diagram for the execution of instruction cycle



CONCLUSION

The **Instruction Execution Cycle** is the core process that enables the CPU to execute programs systematically. Improving each stage of the cycle increases processing speed, reduces delays, and enhances overall CPU performance, leading to faster and more efficient computer systems.

Q3. 8085 vs 8086 Architecture

Explanation

The 8085 is an 8-bit microprocessor with a simple register set (A, B, C, D, E, H, L) and supports immediate, direct, register, and register-indirect addressing modes, but has no memory segmentation, which limits its addressable memory and multitasking ability. The 8086 is a 16-bit microprocessor with general-purpose registers (AX, BX, CX, DX) plus dedicated segment registers (CS, DS, SS, ES), enabling memory segmentation and access to a full 1 MB address space.

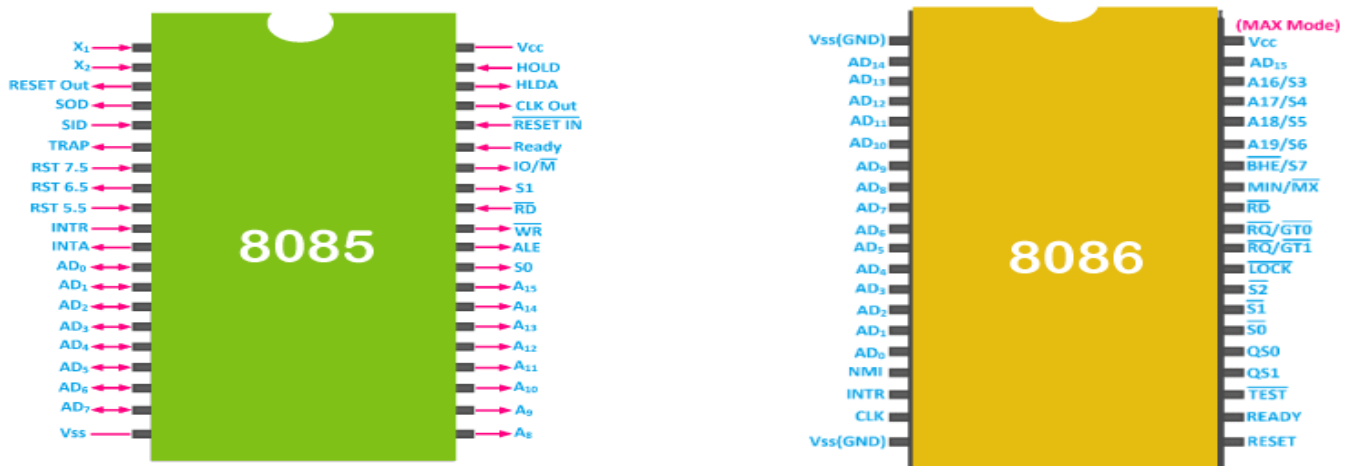
This segmentation allows the 8086 to isolate code, data, and stack areas, supporting safer and more efficient multitasking and larger real-time applications, whereas the 8085's flat, unsegmented memory model restricts it to simpler, smaller embedded control tasks. These architectural differences directly influence program execution efficiency and the complexity of applications each processor can reliably handle.

Relationship

- Both **8085** and **8086** are Intel microprocessors designed to execute instructions and control computer operations.
- The **8086** is an advanced version of the **8085**, offering higher processing speed and improved performance.
- 8085 is an **8-bit processor**, while 8086 is a **16-bit processor** with a wider data bus.
- The 8086 architecture introduces **segmented memory** and a **Bus Interface Unit (BIU)** and **Execution Unit (EU)**, making it more efficient than the 8085.
- Both architectures include an **ALU**, **registers**, **control unit**, and **instruction decoder**, but the 8086 provides more registers and a larger memory addressing capability.

Concept map

Difference Between 8085 and 8086 Microprocessor



Conclusion

The **8086** architecture is a major advancement over the **8085**, offering better performance, higher processing speed, larger memory capacity, and efficient instruction execution. While the **8085** is suitable for basic embedded applications, the **8086** is ideal for more powerful computing systems.

Q4. Addressing Modes and Instruction Types

Explanation

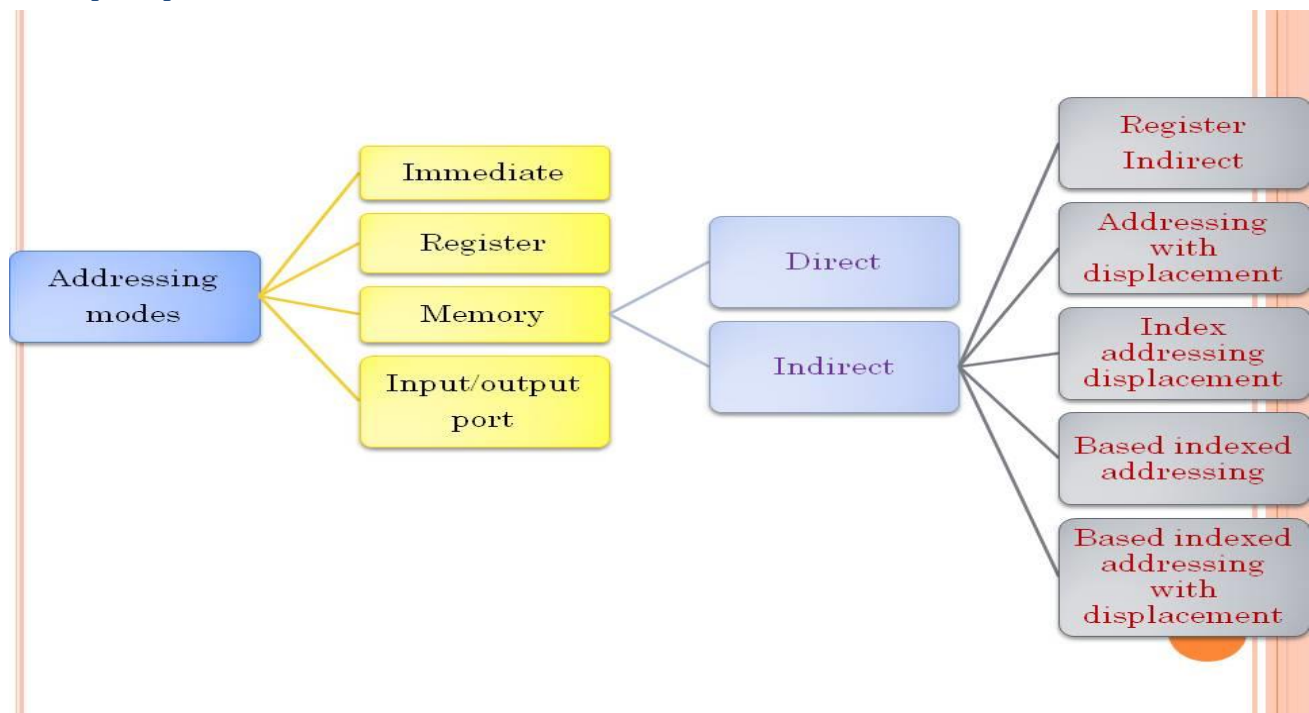
Addressing modes – immediate, direct, register, indirect, and indexed – determine how an instruction locates its operand, and each is naturally suited to certain instruction types. Immediate and register addressing suit arithmetic instructions since operands are readily available with minimal delay. Direct and control-flow addressing suit data transfer and control instructions that reference fixed memory locations or jump targets. Indirect and indexed addressing suit data transfer instructions involving pointers or arrays.

These choices affect memory access time (indirect/indexed modes need an extra memory reference), instruction size (immediate/direct modes often need extra operand bytes), and execution speed (register addressing is fastest). In practice, constants use immediate addressing, array/loop processing uses indexed addressing, and pointer-based data structures use indirect addressing.

Relationship

- **Addressing modes** determine how the CPU locates operands (data) required by an instruction.
- **Instruction types** define the operation to be performed, such as data transfer, arithmetic, logical, branching, and machine control.
- Every instruction uses an appropriate addressing mode to access data efficiently.
- The choice of addressing mode affects the speed, memory usage, and flexibility of instruction execution.
- Together, addressing modes and instruction types enable the CPU to execute programs accurately and efficiently.

Concept map



CONCLUSION

Addressing modes and **instruction types** are fundamental to microprocessor operation. Addressing modes provide efficient ways to access data, while instruction types define the required operations. Together, they improve program execution, optimize memory access, and enhance the overall performance of the processor.

Q5. Number Systems, Instructions, Memory and Debugging

Explanation

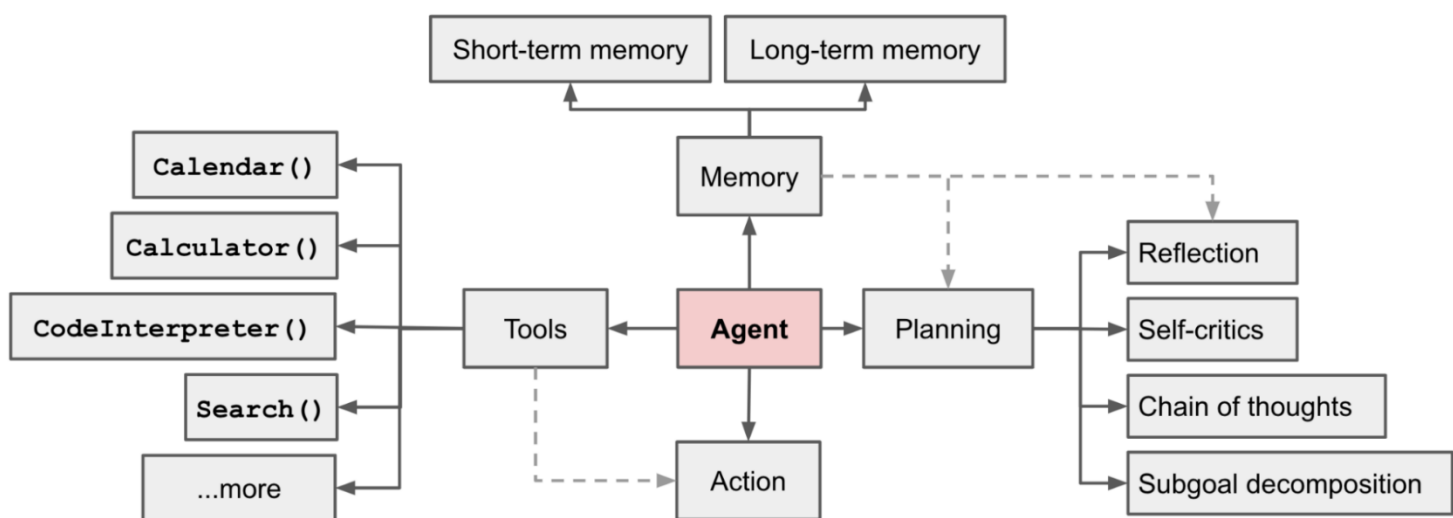
Binary is the native language of the CPU, while hexadecimal is a compact, human-readable shorthand where each digit maps to exactly 4 bits. This conversion links directly to how instructions are represented and stored in memory (as binary opcodes and operands) and how engineers debug systems, since reading long binary strings is error-prone while hexadecimal is far easier to scan and interpret.

Because the CPU stores and interprets all data in binary, but humans work more efficiently in hexadecimal during low-level programming and system design, hexadecimal representation has become the standard for displaying memory dumps, machine code, and register contents in debuggers and datasheets.

Relationship

- Number systems (Binary, Decimal, Hexadecimal, Octal) are used to represent data and addresses in a computer.
- Instructions tell the CPU what operations to perform on the data.
- Memory stores program instructions, data, and intermediate results during execution.
- Debugging helps identify and correct errors in programs by examining instructions, memory contents, and program execution.
- Together, number systems, instructions, memory, and debugging ensure accurate program execution and efficient system performance.

Concept map



CONCLUSION

- **Number systems, instructions, memory, and debugging** are essential components of computer programming and microprocessor systems. Number systems represent data, instructions perform operations, memory stores programs and data, and debugging ensures error-free execution. Together, they improve the reliability, accuracy, and efficiency of computer systems.